

REQUIREMENTS SPECIFICATIONS

UNIVERSITY CORRIDOR MEETING ROOM AND WORKSPACE RESERVATION SYSTEM

TEAM: - SYED MEESUM RAZA NAQVI (24100183)

STUDENT ID	NAME
24100183	SYED MEESUM RAZA NAQVI

TABLE OF CONTENTS

1.	Introduction.....	3
	System Actors	4
2.	Use Cases	5
2.1	Use Case Diagrams.....	5
2.2	Description of Use Cases.....	5
2.2.1 to 2.2.20		5
3.	Class Diagram	16
3.1	Diagram	117
3.2	Description	18
4.	Sequence Diagrams.....	22
4.1 to 4.20		22
5.	State Diagrams	32
5.1 to 5.10		32
6.	Non-functional Requirements / Quality Attributes.....	36
7.	Who Did What?	37
8.	Review checklist	37

1. Introduction

<Give an overview of the project here. The overview must highlight the overall objectives of the project and its potential users and customers.>

Overview:

The University Corridor Meeting Room and Workspace Reservation System makes it easier to book meeting rooms and individual workspaces in different university departments. Each department has several corridors, and each corridor contains multiple rooms. Some of these rooms have several workspaces to meet different needs. Workspaces are of different types too. The individual workspace for like studying alone, shared workspace for group studying or office hours. Other rooms may be reserved exclusively for faculty meetings and do not have multiple workspaces; instead, the entire room will be booked for a specified time. The system helps manage these spaces better, reduces booking conflicts, and makes sure the rooms are used efficiently.

Objectives:

- To provide a centralized platform for booking meeting rooms and workspaces.
- To allow users to view, reserve, and manage bookings efficiently.
- To prevent double bookings and enforce booking policies to maintain fair usage.

Potential Users:

- Faculty members for departmental meetings.
- Teaching Assistants (TAs) for office hours and student consultations.
- Senior Year Students for studying alone or in group for project (restricted to 5 members).
- Application system for handling booking, conflict resolution, and notification.
- Admin for creating user accounts based on the user's status.

System Actors

<List down the actor names and give a 2-3 lines description of the role of each actor>

Actor Name	Description
Faculty Member	A user who can view, reserve, and manage bookings for both meeting rooms and all types of workspaces.
Teaching Assistant (TA)	A user with the ability to reserve either individual workspaces or shared workspaces for student consultations or meetings (i.e., office hours)
(Senior) Student	A user who can book individual workspaces for study or shared workspace for project work (restricted to 5 members only)
System	The application that handles booking, conflict resolution, and notification.
Admin	The admin is responsible for creating user accounts based on the user's status, such as Faculty Member, Teaching Assistant (TA), or Senior Student.

2. Use Cases

2.1 Use Case Diagrams

<Use standard UML notation>



2.2 Description of Use Cases

[Select 20 most important use cases of your project and create their comprehensive descriptions.]

<Write description of each use case separately using the template below.>

2.2.1 View Available Rooms and Workspaces

Identifier	UC-001
Purpose	Allow users to see which meeting rooms and workspaces are available.
Pre-conditions	User must be logged into the system.
Post-conditions	System displays a list of available rooms and workspaces based on the user's criteria.
Step #	Typical Course of Action
1.	User logs into the system.
2.	User navigates to the 'View Availability' page.
3.	System prompts the user to select criteria (date, time, location, type).
4.	User inputs the criteria.
5.	System displays available rooms and workspaces.
Step #	Alternate Courses of Action
1.	If no rooms or workspaces match the criteria, the system displays a 'No availability' message and suggests the closest available options.

Step #	Exception Paths
1.	If the system is unable to retrieve availability due to a database connection error, an error message is displayed, and the user is prompted to try again later.

2.2.2 Reserve a Room or Workspace

Identifier	UC-002
Purpose	Enable users to reserve a specific room or workspace.
Pre-conditions	Room or workspace must be available, and the user must be logged in.
Post-conditions	Selected room or workspace is booked, and confirmation is sent to the user.
Step #	Typical Course of Action
1.	User logs into the system.
2.	User selects a room or workspace from the availability list.
3.	User selects the desired time slot.
4.	System verifies the availability of the room/workspace.
5.	System books the room/workspace and sends a confirmation notification to the user.
Step #	Alternate Courses of Action
1.	If the room/workspace becomes unavailable during the booking process, the system informs the user and suggests alternative options.
Step #	Exception Paths
1.	If the system experiences a booking conflict due to simultaneous requests, it informs the user of the conflict and cancels the booking attempt, suggesting alternative times or rooms.

2.2.3 Cancel a Reservation

Identifier	UC-003
Purpose	Allow users to cancel an existing reservation.
Pre-conditions	User must have an existing reservation.

Post-conditions	Reservation is canceled, and room/workspace is made available for others.
Step #	Typical Course of Action
1.	User logs into the system.
2.	User navigates to their reservation list.
3.	User selects the reservation to cancel.
4.	System confirms the cancellation.
5.	System updates the availability and sends a notification to the user.
Step #	Alternate Courses of Action
1.	If the user attempts to cancel a reservation that does not exist, the system displays an error message.
Step #	Exception Paths
1.	If the system cannot process the cancellation due to a technical issue, it displays an error message and instructs the user to contact support.

2.2.4 Receive Notifications

Identifier	UC-004
Purpose	Inform users about their booking status.
Pre-conditions	User must have a booking or be attempting to make a booking.
Post-conditions	User is notified about the status of their booking.
Step #	Typical Course of Action
1.	System sends notifications for booking confirmation, changes, or cancellations.
2.	User receives a notification via email or the system dashboard.
Step #	Alternate Courses of Action
1.	If the user does not receive the notification via email, the system ensures it appears on the user dashboard.
Step #	Exception Paths
1.	If the notification service is down, the system logs the failure and attempts to resend the notification later.

2.2.5 Conflict Resolution

Identifier	UC-005
Purpose	System to handle conflicts arising from simultaneous booking requests.
Pre-conditions	Multiple users attempt to book the same room or workspace at the same time.
Post-conditions	Conflict is resolved, and the room/workspace is booked by one user.
Step #	Typical Course of Action
1.	System detects conflicting booking requests.
2.	System applies predefined rules (e.g., first-come-first-served).
3.	System grants the booking to one user and sends a notification to the others.
Step #	Alternate Courses of Action
1.	If no resolution can be achieved, the system informs all users involved and prompts them to select alternative options.
Step #	Exception Paths
1.	If the conflict resolution module fails, the system temporarily holds all bookings and escalates the issue to an administrator for manual intervention.

2.2.6 Update User Profile

Identifier	UC-006
Purpose	Allow users to update their profile information.
Pre-conditions	User must be logged into the system.
Post-conditions	User profile is updated.
Step #	Typical Course of Action
1.	User logs into the system.
2.	User navigates to the profile page.
3.	User updates their information.
4.	System saves the updated profile.
Step #	Alternate Courses of Action

1.	If some fields are left empty or contain invalid data, the system prompts the user to correct the errors before saving.
Step #	Exception Paths
1.	If the system fails to save changes due to a database issue, it displays an error message and logs the error for further investigation.

2.2.7 View User Booking History

Identifier	UC-007
Purpose	Allow users to view their past and upcoming bookings.
Pre-conditions	User must be logged into the system.
Post-conditions	User views a list of past and upcoming bookings.
Step #	Typical Course of Action
1.	User logs into the system.
2.	User navigates to the booking history page.
3.	System displays a list of the user's past and upcoming bookings.
Step #	Alternate Courses of Action
1.	If the user has no booking history, the system displays a message indicating no bookings found.
Step #	Exception Paths
1.	If the system cannot retrieve booking history due to a server error, it displays an error message and logs the issue for technical review.

2.2.8 Search for Specific Room or Workspace

Identifier	UC-008
Purpose	Allow users to search for a specific room or workspace.
Pre-conditions	User must be logged into the system.
Post-conditions	System displays search results matching the user's query.

Step #	Typical Course of Action
1.	User logs into the system.
2.	User enters search criteria (e.g., room name, location).
3.	System displays matching rooms or workspaces.
Step #	Alternate Courses of Action
1.	If no rooms or workspaces match the search criteria, the system displays a 'No results found' message.
Step #	Exception Paths
1.	If the search functionality fails due to a query processing error, the system displays an error message and suggests using basic filters.

2.2.9 Set Recurring Reservations

Identifier	UC-009
Purpose	Allow users (faculty, Tas) to set recurring reservations for regular meetings or sessions.
Pre-conditions	Room or workspace must be available for the desired recurring schedule.
Post-conditions	Recurring reservation is set, and confirmation is sent to the user.
Step #	Typical Course of Action
1.	User logs into the system.
2.	User selects a room/workspace and sets the recurrence rules (e.g., daily, weekly).
3.	System checks availability for each instance of the recurrence.
4.	System books the room/workspace and sends a confirmation.
Step #	Alternate Courses of Action
1.	If some recurrence slots are unavailable, the system suggests modifications or alternative recurrence schedules.
Step #	Exception Paths
1.	If the system fails to process the recurring reservation, it displays an error message and logs the failure for technical investigation.

2.2.10 Approve or Reject Booking Requests

Identifier	UC-010
Purpose	System to automatically approve or reject booking requests based on predefined rules.
Pre-conditions	Booking request is submitted.
Post-conditions	Booking is either approved or rejected, and the user is notified.
Step #	Typical Course of Action
1.	User submits a booking request.
2.	System checks the request against predefined rules.
3.	System approves or rejects the request.
4.	System sends a notification to the user.
Step #	Alternate Courses of Action
1.	If the system cannot determine approval based on rules, it flags the request for manual review.
Step #	Exception Paths
1.	If the system fails to apply approval rules due to a configuration error, it logs the error and temporarily halts booking processing.

2.2.11 Provide Booking Recommendations

Identifier	UC-011
Purpose	System to suggest alternative rooms or workspaces if the preferred one is unavailable.
Pre-conditions	User attempts to book an unavailable room/workspace.
Post-conditions	User is provided with alternative booking options.
Step #	Typical Course of Action
1.	User selects an unavailable room/workspace.
2.	System suggests alternative rooms/workspaces.
3.	User selects an alternative option.

Step #	Alternate Courses of Action
1.	If no alternatives are available, the system displays a 'No alternatives available' message.
Step #	Exception Paths
1.	If the recommendation engine fails, the system displays a general error message and suggests contacting support.

2.2.12 Generate Booking Reports

Identifier	UC-012
Purpose	System to generate reports on room and workspace usage.
Pre-conditions	System must have booking data.
Post-conditions	Usage report is generated and accessible to authorized users.
Step #	Typical Course of Action
1.	System compiles booking data.
2.	System generates a usage report.
3.	Report is made available to administrators.
Step #	Alternate Courses of Action
1.	If the report takes too long to generate, the system informs the user.
Step #	Exception Paths
1.	If the report generation fails due to a data access issue, the system logs the error and notifies the administrator.

2.2.13 Send Reminders for Upcoming Reservations

Identifier	UC-013
Purpose	System to send reminders to users about their upcoming reservations.
Pre-conditions	User has an upcoming reservation.
Post-conditions	Reminder is sent to the user.

Step #	Typical Course of Action
1.	System identifies upcoming reservations.
2.	System sends reminders to the respective users.
3.	Users receive reminders via email or the system dashboard.
Step #	Alternate Courses of Action
1.	If the user has disabled notifications, the system does not send the reminder but marks it as sent.
Step #	Exception Paths
1.	If the reminder fails to send due to a network issue, the system logs the failure and retries after a set interval.

2.2.14 Log User Actions

Identifier	UC-014
Purpose	System to keep a log of user actions for auditing purposes.
Pre-conditions	User performs an action related to bookings.
Post-conditions	Action is logged in the system.
Step #	Typical Course of Action
1.	User performs a booking-related action.
2.	System logs the action with a timestamp and user details.
Step #	Alternate Courses of Action
1.	If the action log is full, the system archives old logs to create space for new entries.
Step #	Exception Paths
1.	If the logging system is down, the system queues actions for logging once the service is restored.

2.2.15 View Room and Workspace Details

Identifier	UC-015
Purpose	Allow users to view details of a room or workspace before booking.

Pre-conditions	User must be logged into the system.
Post-conditions	User views detailed information about the selected room or workspace.
Step #	Typical Course of Action
1.	User logs into the system.
2.	User navigates to the list of available rooms or workspaces.
3.	User selects a room/workspace to view details.
4.	System displays detailed information about the selected room/workspace.
Step #	Alternate Courses of Action
1.	If the user navigates away without viewing details, the system returns them to the main list.
Step #	Exception Paths
1.	If the system fails to retrieve details, an error message is displayed, and the user is prompted to try again.

2.2.16 Set Maximum Occupancy Rules

Identifier	UC-016
Purpose	System to enforce rules on the maximum number of occupants per room or workspace.
Pre-conditions	Maximum occupancy rules are defined in the system.
Post-conditions	Booking requests exceeding occupancy limits are rejected.
Step #	Typical Course of Action
1.	System checks the number of occupants during a booking request.
2.	If the request exceeds the limit, the system denies the booking.
3.	System informs the user of the occupancy rule.
Step #	Alternate Courses of Action
1.	If the booking request is within the limit, the system proceeds with the booking.
Step #	Exception Paths

1.	If the occupancy data is not available, the system defaults to a standard limit and logs the error for further investigation.
----	---

2.2.17 Access Support for Booking Issues

Identifier	UC-017
Purpose	Provide support for users facing booking issues.
Pre-conditions	User must be logged into the system.
Post-conditions	User accesses support and gets assistance.
Step #	Typical Course of Action
1.	User logs into the system.
2.	User navigates to the support page.
3.	User describes the issue.
4.	System logs the issue and routes it to the support team.
Step #	Alternate Courses of Action
1.	If the issue is common, the system suggests an FAQ or automated solution.
Step #	Exception Paths
1.	If the support service is down, the system displays a notice and provides alternative contact information.

2.2.18 Notify Users of System Maintenance

Identifier	UC-018
Purpose	System to notify users about upcoming system maintenance.
Pre-conditions	Maintenance is scheduled.
Post-conditions	Users are informed about the maintenance schedule.
Step #	Typical Course of Action
1.	System identifies users affected by maintenance.
2.	System sends notifications about the maintenance schedule.

3.	Users receive notifications via email or the system dashboard.
Step #	Alternate Courses of Action
1.	If maintenance is canceled, the system sends an update notification.
Step #	Exception Paths
1.	If the notification fails, the system logs the issue and attempts to resend later.

2.2.19 Rate and Provide Feedback on Rooms/Workspaces

Identifier	UC-019
Purpose	Allow users to rate and provide feedback on rooms and workspaces.
Pre-conditions	User must have a past booking for the room or workspace.
Post-conditions	Feedback and rating are submitted and stored in the system.
Step #	Typical Course of Action
1.	User logs into the system.
2.	User navigates to the past bookings.
3.	User selects a room/workspace and submits a rating and feedback.
4.	System stores the feedback.
Step #	Alternate Courses of Action
1.	If the user leaves some feedback fields blank, the system prompts for optional completion.
Step #	Exception Paths
1.	If the feedback submission fails, the system displays an error message and suggests trying again later.

2.2.20 Access Booking Statistics Dashboard

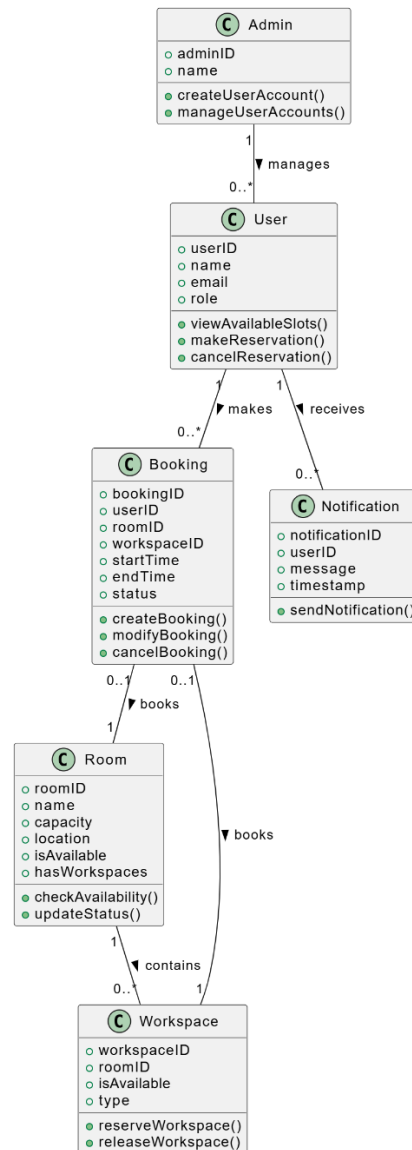
Identifier	UC-020
Purpose	System to provide administrators with access to booking statistics and usage data.
Pre-conditions	User must have administrative access.

Post-conditions	User views booking statistics and usage data.
Step #	Typical Course of Action
1.	Admin logs into the system.
2.	Admin navigates to the statistics dashboard.
3.	System displays booking statistics and usage data.
Step #	Alternate Courses of Action
1.	If the admin selects specific filters, the system displays filtered data.
Step #	Exception Paths
1.	If the statistics module is down, the system displays an error message and logs the issue for technical review.

3. Class Diagram

3.1 Diagram

<Use standard UML notation>



3.2 Description

<Give brief description/purpose of each class in the class diagram. Give readable names to classes, attributes and operations.>

1) Admin Class

- **Purpose:** The `Admin` class represents administrative users who have the authority to manage other users within the system. Admins are responsible for creating and managing user accounts, ensuring that users are set up correctly based on their roles (Faculty, Teaching Assistants, Senior Students). This class encapsulates the functionalities needed for user account management and oversight.
- **Attributes:**
`adminID`: A unique identifier for each admin.
`name`: The name of the admin.
- **Methods:**
`createUserAccount()`: Creates new user accounts.
`manageUserAccounts()`: Modifies or deletes existing user accounts, including updating user information and roles.

2) User Class

- **Purpose:** The `User` class represents all types of users in the system, including faculty members, teaching assistants, and senior students. This class captures common attributes and behaviors for these users, such as viewing available resources, making reservations, and managing bookings.
- **Attributes:**
`userID`: A unique identifier for each user.
`name`: The name of the user.
`email`: The user's email address.
`role`: The role of the user, such as Faculty, TA, or Senior Student.
- **Methods:**
`viewAvailableSlots()`: Allows users to see available rooms and workspaces.
`makeReservation()`: Enables users to make a booking for a room or workspace.
`cancelReservation()`: Allows users to cancel an existing reservation.

3) Booking Class

- **Purpose:** The `Booking` class handles the reservation process for rooms and workspaces. It keeps track of booking details, such as which user made the booking, what room or workspace is reserved, and the time and status of the reservation.
- **Attributes:**
`bookingID`: A unique identifier for each booking.
`userID`: The ID of the user who made the booking.

``roomId``: The ID of the booked room (nullable if a workspace is booked).
``workspaceID``: The ID of the booked workspace (nullable if a whole room is booked).
``startTime``: The starting time of the booking.
``endTime``: The ending time of the booking.
``status``: The status of the booking (e.g., confirmed, canceled).

- **Methods:**

``createBooking()``: Creates a new booking.
``modifyBooking()``: Modifies an existing booking.
``cancelBooking()``: Cancels an existing booking.

4) Notification Class

- **Purpose:** The ``Notification`` class is responsible for managing notifications sent to users regarding their bookings. It helps keep users informed about the status of their bookings, reminders, or any changes that may occur.
- **Attributes:**
 - ``notificationID``: A unique identifier for each notification.
 - ``userID``: The ID of the user receiving the notification.
 - ``message``: The content of the notification.
 - ``timestamp``: The time when the notification was sent.
- **Methods:**
 - ``sendNotification()``: Sends a notification to the user.

5) Room Class

- **Purpose:** The ``Room`` class represents the physical meeting rooms available for booking within the system. It includes attributes that describe each room's properties and methods to check availability and update the room's status.
- **Attributes:**
 - ``roomId``: A unique identifier for each room.
 - ``name``: The name of the room.
 - ``capacity``: The maximum number of occupants the room can accommodate.
 - ``location``: The location of the room.
 - ``isAvailable``: A flag indicating whether the room is currently available for booking.
 - ``hasWorkspaces``: A flag indicating if the room contains individual workspaces.
- **Methods:**
 - ``checkAvailability()``: Checks if the room is available for booking.

``updateStatus()``: Updates the room's availability status.

6) Workspace Class

- **Purpose:** The ``Workspace`` class represents individual workspaces within a room that can be reserved separately. This class helps manage the availability and reservation of smaller workspaces within larger rooms, catering to different needs (individual study vs. group meetings).
- **Attributes:**
 - ``workspaceID``: A unique identifier for each workspace.
 - ``roomID``: The ID of the room that contains the workspace.
 - ``isAvailable``: A flag indicating whether the workspace is available for booking.
 - ``type``: The type of workspace (e.g., individual, shared).
- **Methods:**
 - ``reserveWorkspace()``: Reserves the workspace for a user.
 - ``releaseWorkspace()``: Releases the workspace, making it available for others.

Summary of the Class Diagram

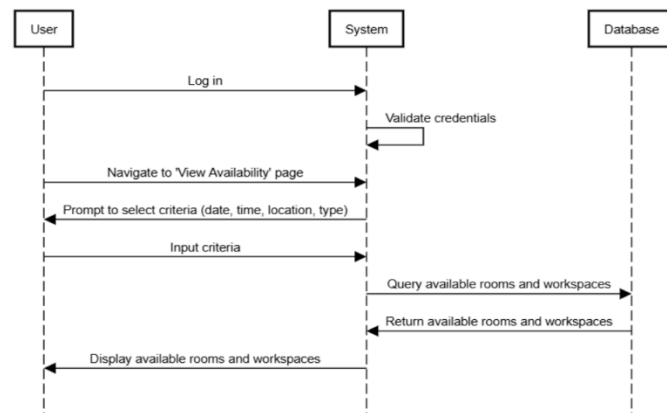
- **Admin:** Manages the ``User`` class, which includes creating and managing user accounts.
- **User:** Interacts with the system by viewing availability, making reservations, and canceling bookings.
- **Booking:** Connects ``User`` with ``Room`` or ``Workspace``, handling the reservation details.
- **Notification:** Keeps ``User`` informed about their booking status or changes.
- **Room:** Represents physical rooms available for booking, with capabilities to check availability and update status.
- **Workspace:** Represents smaller areas within a room, individually bookable and managed for availability.

4. Sequence Diagrams

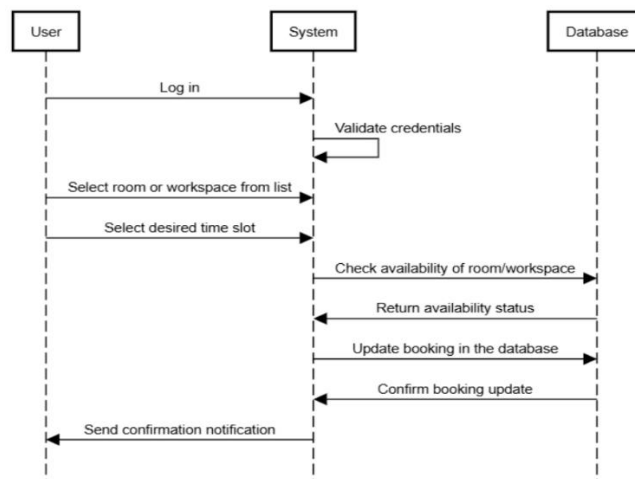
<Create a sequence diagram for each of the 20 use cases selected above.>

4.1 Use case Name: View Available Rooms and Workspaces

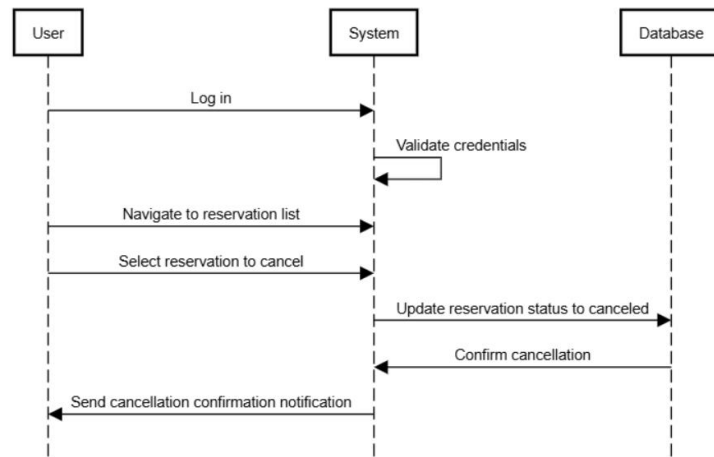
<Draw the sequence diagram of the use case using standard UML notation>



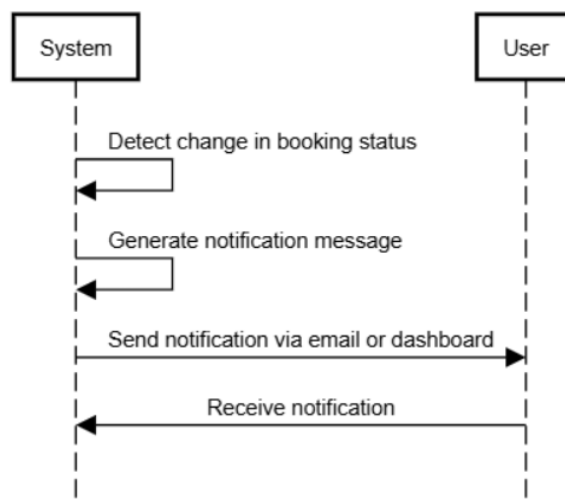
4.2 Reserve a Room or Workspace



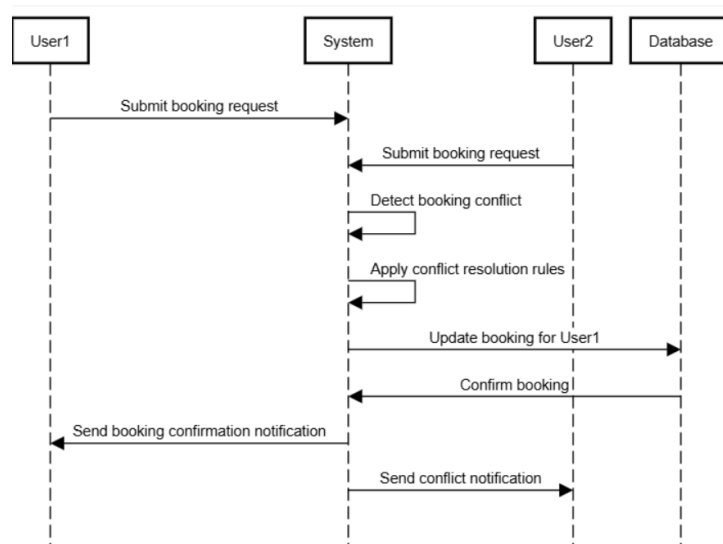
4.3 Cancel a Reservation



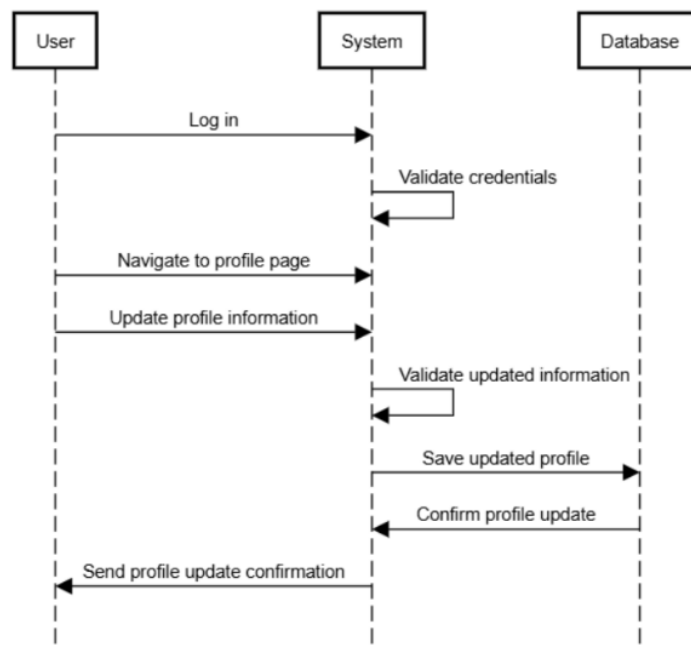
4.4 Receive Notifications



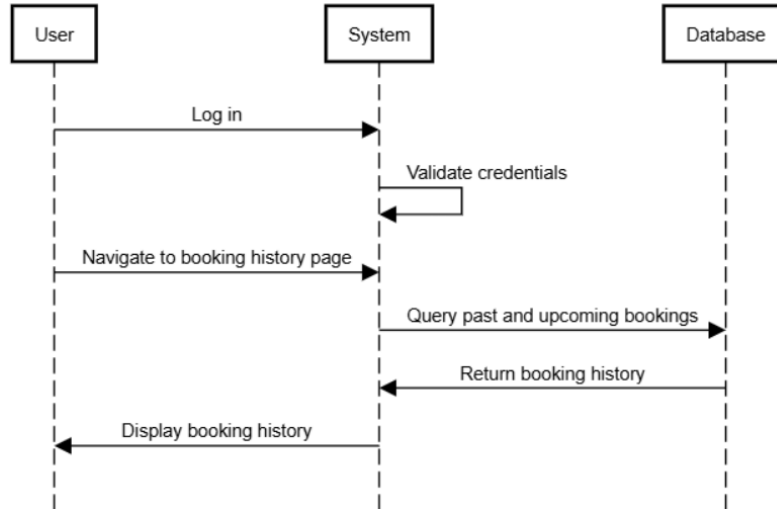
4.5 Conflict Resolution



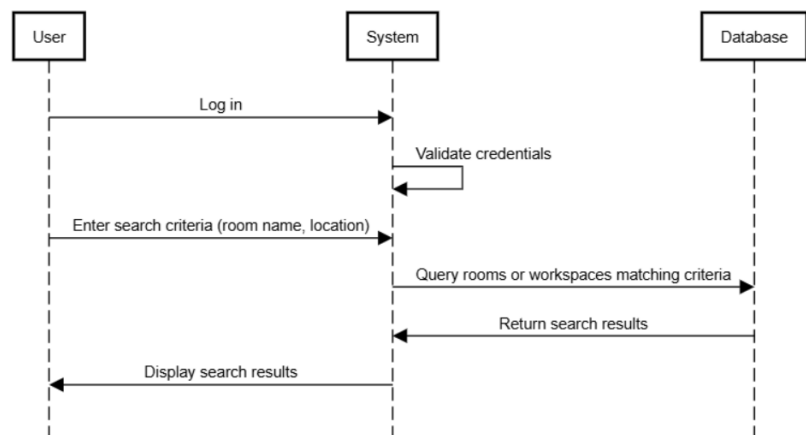
4.6 Update User Profile



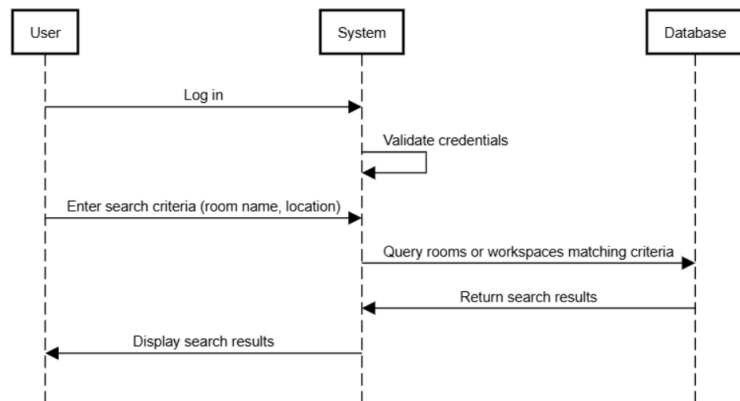
4.7 View User Booking History



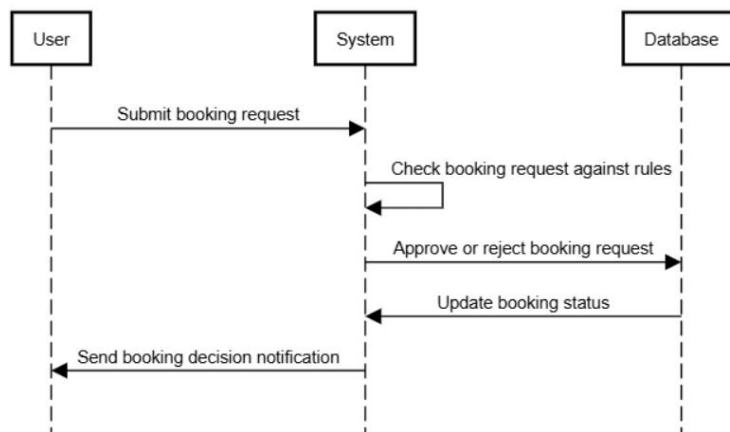
4.8 Search for Specific Room or Workspace



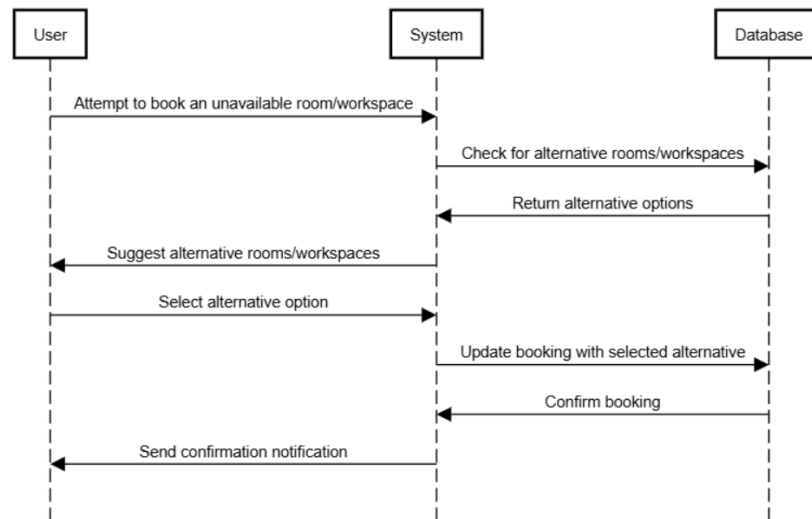
4.9 Set Recurring Reservations



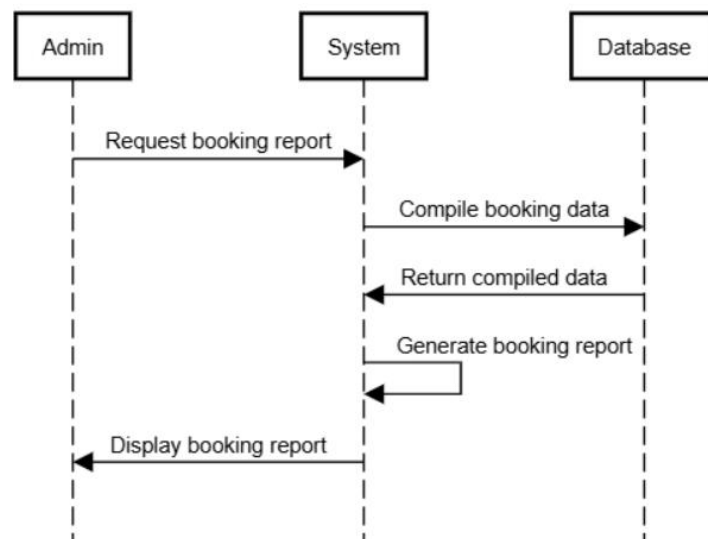
4.10 Approve or Reject Booking Requests



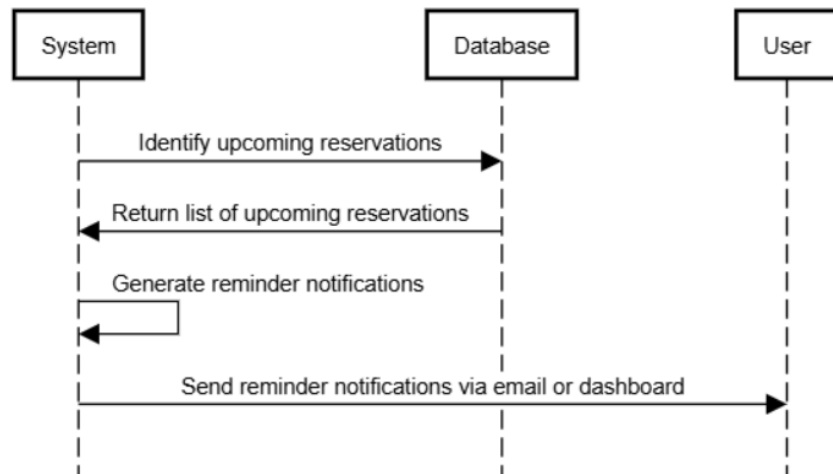
4.11 Provide Booking Recommendations



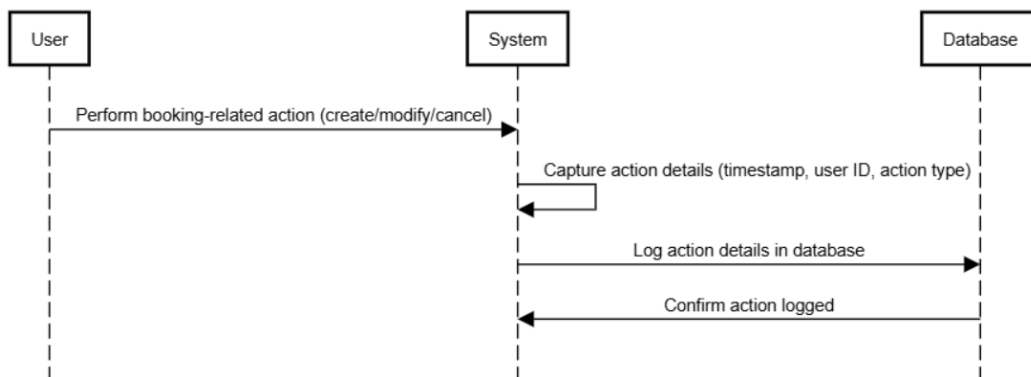
4.12 Generate Booking Reports



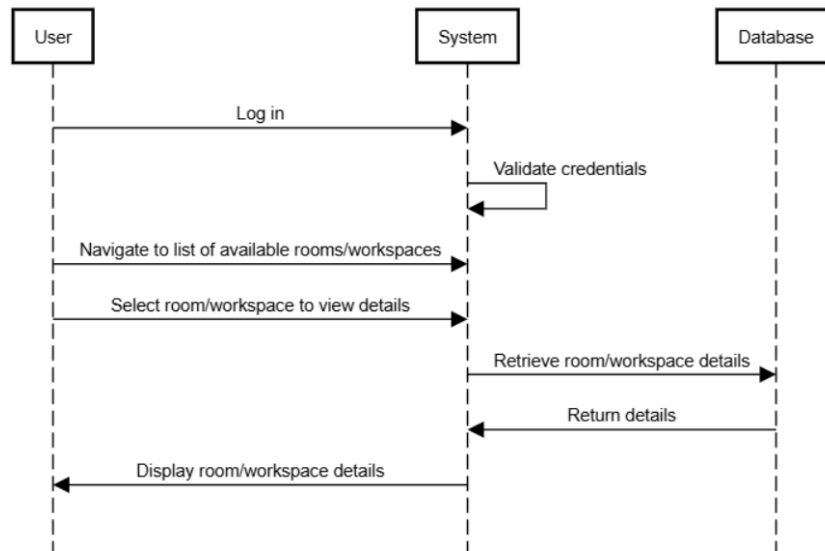
4.13 Send Reminders for Upcoming Reservations



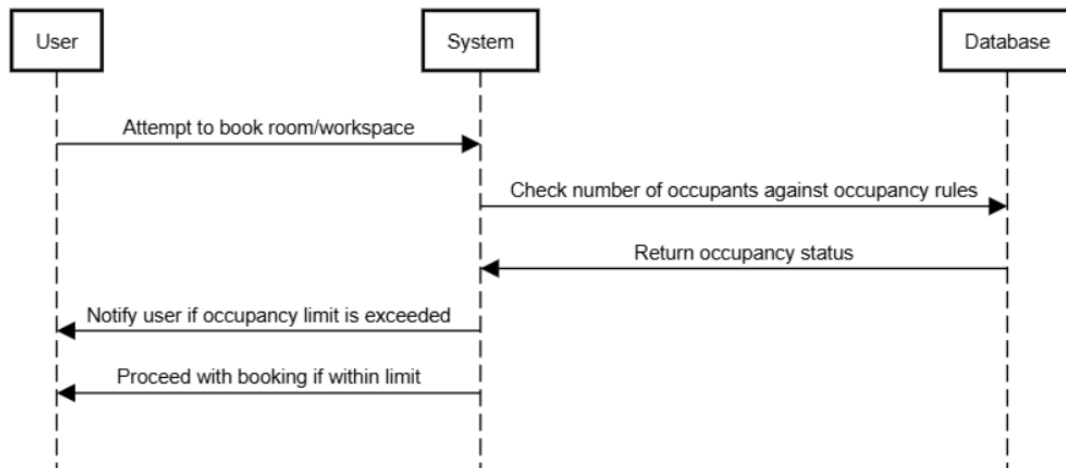
4.14 Log User Actions



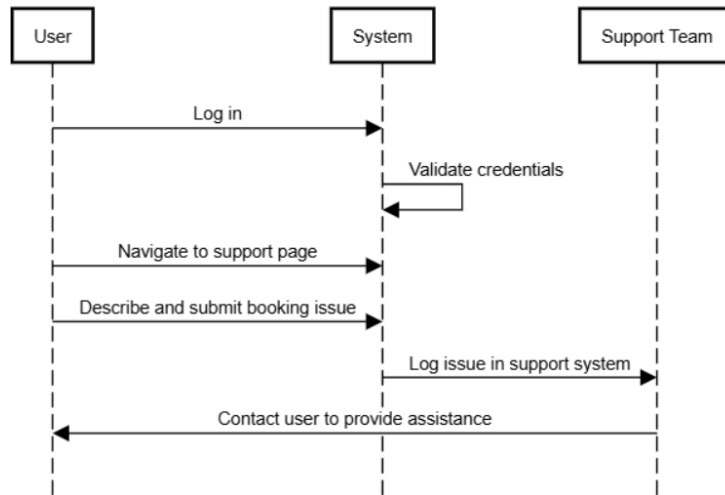
4.15 View Room and Workspace Details



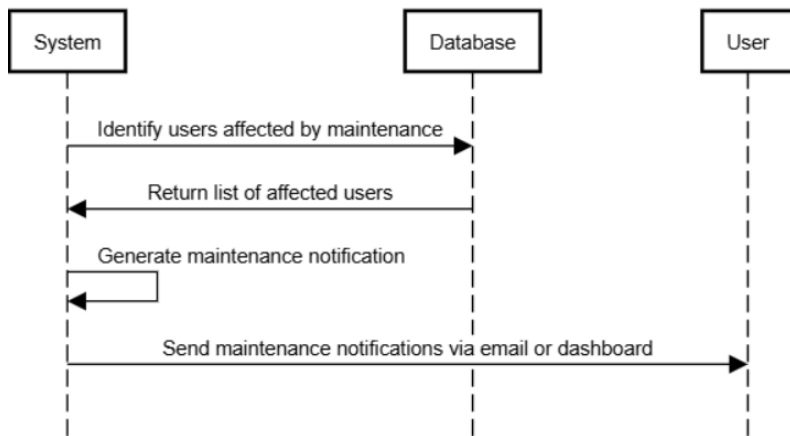
4.16 Set Maximum Occupancy Rules



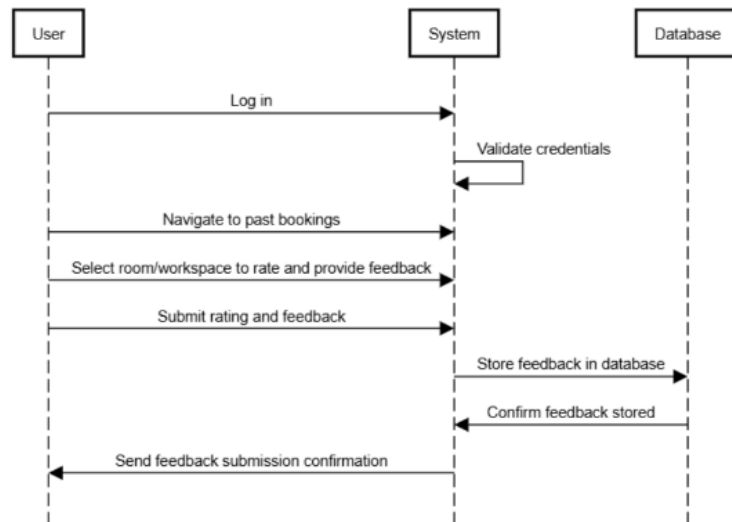
4.17 Access Support for Booking Issues



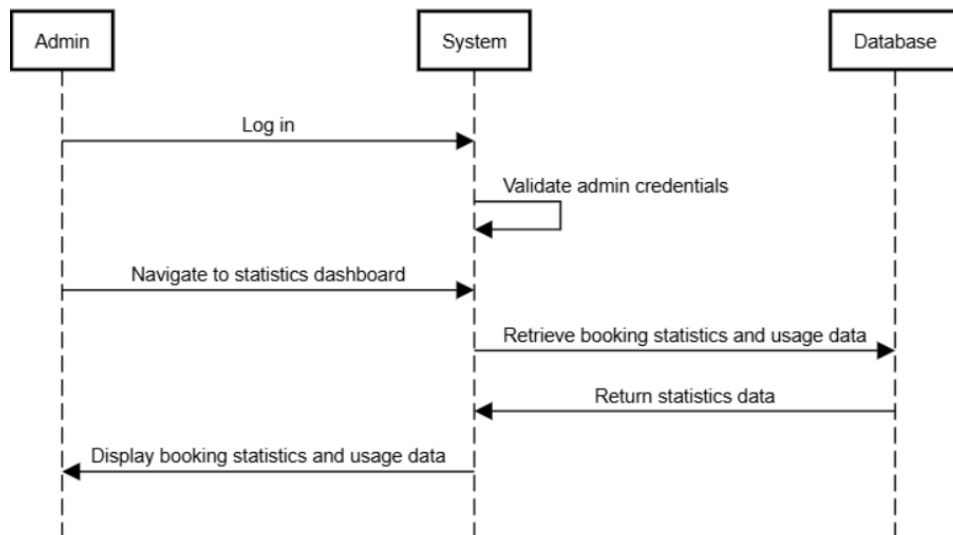
4.18 Notify Users of System Maintenance



4.19 Rate and Provide Feedback on Rooms/Workspaces



4.20 Access Booking Statistics Dashboard



5. State Diagrams

<Repeat the following if you need to draw state diagrams of multiple objects>

5.1 Diagram details

<Name of the object whose state is being shown along with details of states and messages.>

Name: Booking Object

Description:

- **States:**

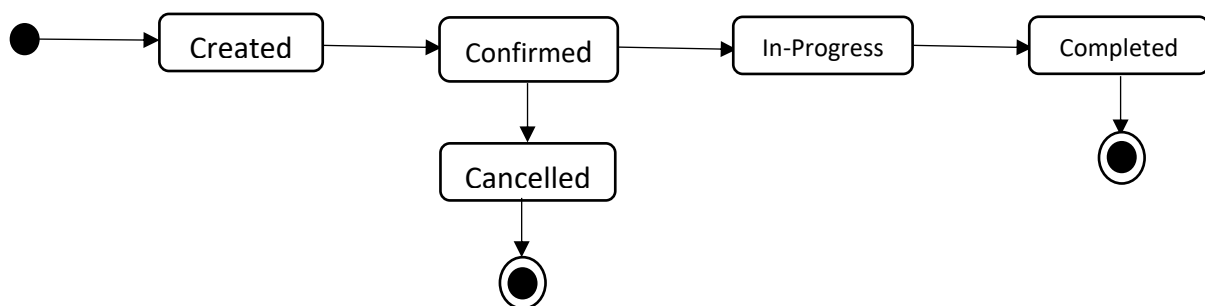
1. **Created:** A booking is created but not yet confirmed.
2. **Confirmed:** The booking has been confirmed and is valid.
3. **In Progress:** The time for the booking has arrived, and the room/workspace is currently in use.
4. **Completed:** The booking has ended, and the room/workspace is free again.
5. **Cancelled:** The booking was cancelled before the scheduled time.

Transitions:

- From **Created** to **Confirmed**: When the user confirms the booking.
- From **Confirmed** to **In Progress**: When the booked time starts.
- From **In Progress** to **Completed**: When the booked time ends.
- From **Confirmed** to **Cancelled**: If the user cancels the booking before the booking starts.

5.2 Diagram

<Use standard UML notation>



5.3 Diagram details

Name: Room Object

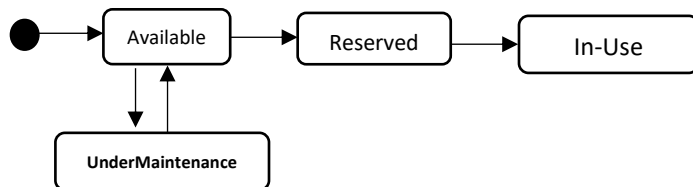
Description:

- **States:**
 1. **Available:** The room is free and can be booked.
 2. **Reserved:** The room has been booked but is not yet in use.
 3. **In Use:** The room is currently being used during a booked time.
 4. **Under Maintenance:** The room is not available for booking due to maintenance.

Transitions:

- **From Available to Reserved:** When a user successfully books the room.
- **From Reserved to In Use:** When the booking time starts.
- **From In Use to Available:** When the booking ends, and the room becomes free.
- **From Available to Under Maintenance:** When the room is set for maintenance.

5.4 Diagram



5.5 Diagram details

Name: Workspace Object

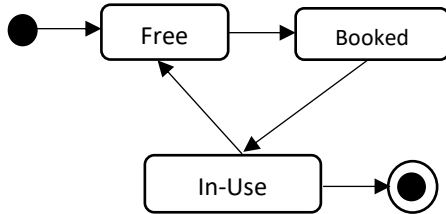
Description:

- **States:**
 1. **Free:** The workspace is available for reservation.
 2. **Booked:** The workspace has been booked but not yet in use.
 3. **In Use:** The workspace is being used during the booked time.

Transitions:

- **From Free to Booked:** When a user books the workspace.
- **From Booked to In Use:** When the booking starts, and the workspace is in use.
- **From In Use to Free:** When the booked time ends.

5.6 Diagram



5.7 Diagram details

Name: User Object

Description:

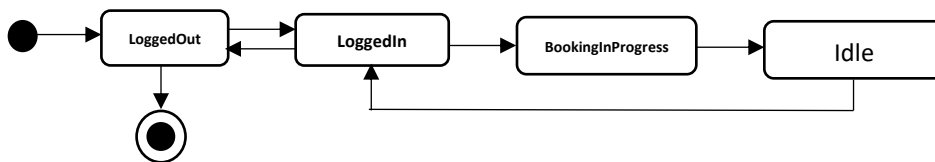
- **States:**

1. **Logged Out:** The user is not logged into the system.
2. **Logged In:** The user has logged into the system and can perform actions such as booking, viewing history, or cancelling a reservation.
3. **Booking In Progress:** The user is in the process of booking a room or workspace.
4. **Idle:** The user is logged in but is not performing any action.

Transitions:

- **From Logged Out to Logged In:** When the user enters valid login credentials.
- **From Logged In to Booking In Progress:** When the user starts the booking process.
- **From Booking In Progress to Idle:** After the booking is confirmed or cancelled.
- **From Logged In to Logged Out:** When the user logs out of the system.

5.8 Diagram



5.9 Diagram details

Name: Notification Object

Description:

- **States:**

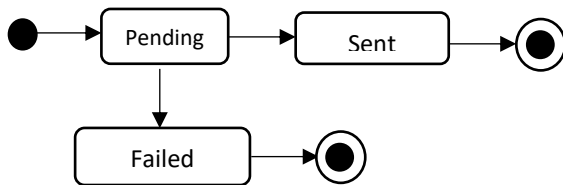
1. **Pending:** A notification is ready to be sent but hasn't been delivered yet.

2. **Sent:** The notification has been successfully delivered to the user.
3. **Failed:** The system was unable to send the notification.

Transitions:

- **From Pending to Sent:** When the notification is successfully delivered.
- **From Pending to Failed:** If the system fails to send the notification after multiple attempts.

5.10 Diagram



6. Non-functional Requirements / Quality Attributes

<Requirements must be testable>

Sr#	Requirements
1	The system must respond to user actions (e.g., viewing availability, making reservations) within 2 seconds 95% of the time.
2	The system must support up to 500 concurrent users without any noticeable performance degradation.
3	The system should be available for use 99.9% of the time over a rolling 30-day period.
4	The system should use less than 1 GB of memory during normal operation.

7. Who Did What?

Name of the Team Member	Tasks done
Syed Meesum Raza Naqvi	Project conception, requirements gathering, UML design, and the rest.

8. Review checklist

Before submission of this deliverable, the team must perform an internal review. Each team member will review one or more sections of the deliverable.

Section Title	Reviewer Name(s)
Introduction	Syed Meesum Raza Naqvi
System Actors	Syed Meesum Raza Naqvi
Use Cases	Syed Meesum Raza Naqvi
Class Diagram	Syed Meesum Raza Naqvi
Sequence Diagrams	Syed Meesum Raza Naqvi
State Diagrams	Syed Meesum Raza Naqvi
Non-Functional Requirements	Syed Meesum Raza Naqvi